

Near-real-time Solutions for Online String Problems

WAAC 2026

Dominik Köppl ¹ Gregory Kucherov ²

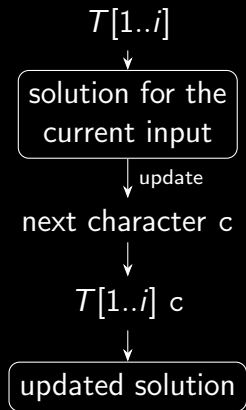
¹: University of Yamanashi

²: Gustave Eiffel University

Background

- Many algorithms are **offline**:
compute solution for the entire input
- While reading the input, we may want to know the solution for the **part read so far**
- i.e., at each point in time, for the text $T[1..i]$, we want to inspect a **snapshot** of the solution currently being maintained
- In an **online algorithm**, whenever a new character arrives, the previous solution is updated efficiently

Here, an *intermediate solution* is not an approximate solution, but the *complete* solution for the prefix received so far.



online algorithms

Definition (online algorithm)

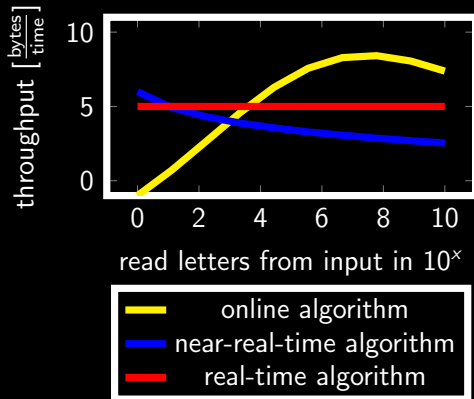
update available solution incrementally as new letters arrive

however:

- most online algorithms are **amortized**: worst-case time per letter may be high!
- real-time** algorithm: worst-case time per letter is a *constant*

here:

- near real-time** algorithm: worst-case time per letter is *logarithmic* to the input size



Real-Time Algorithms

What can be done in real-time?

- ▮ Pattern matching with sliding window Galil'76
- ▮ Palindrome recognition Galil'76
- ▮ Lempel–Ziv 78 factorization Hartman,Rodeh'85

What can be done in near real-time?

- ▮ Suffix tree construction Breslauer,Italiano'13
based on Weiner's algorithm Weiner'73

Weiner's Suffix Tree Construction

input:

- ▮ text $T[1..n]$ of length n over an integer alphabet Σ
- ▮ letters arrive from right to left
- ▮ assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $ST(T)$ of T

goal: construct $ST(T[j..])$ from $ST(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \dots, 1$

Example

($T = \text{aababaa}\$$)

Weiner's Suffix Tree Construction

input:

- ▮ text $T[1..n]$ of length n over an integer alphabet Σ
- ▮ letters arrive from right to left
- ▮ assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $ST(T)$ of T

goal: construct $ST(T[j..])$ from $ST(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \dots, 1$

Example

($T = \text{aababaa}\$$)

- ▮ start with $\$$

$ST(\$)$



Weiner's Suffix Tree Construction

input:

- ▮ text $T[1..n]$ of length n over an integer alphabet Σ
- ▮ letters arrive from right to left
- ▮ assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $ST(T)$ of T

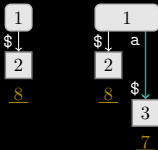
goal: construct $ST(T[j..])$ from $ST(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \dots, 1$

Example

($T = \text{aababaa}\$$)

- ▮ start with $\$$
- ▮ insert $\text{a}\$$

$ST(\$)$ $ST(\text{a}\$)$



Weiner's Suffix Tree Construction

input:

- ▮ text $T[1..n]$ of length n over an integer alphabet Σ
- ▮ letters arrive from right to left
- ▮ assume $T[n] = \$$ is a unique terminal letter

output: suffix tree $ST(T)$ of T

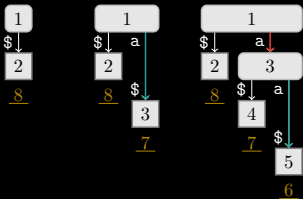
goal: construct $ST(T[j..])$ from $ST(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \dots, 1$

Example

($T = \text{aababaa}\$$)

- ▮ start with $\$$
- ▮ insert $\text{a}\$$
- ▮ insert $\text{aa}\$$

$ST(\$)$ $ST(\text{a}\$)$ $ST(\text{aa}\$)$



Weiner's Suffix Tree Construction

input:

- ▮ text $T[1..n]$ of length n over an integer alphabet Σ
- ▮ letters arrive from right to left
- ▮ assume $T[n] = \$$ is a unique terminal letter

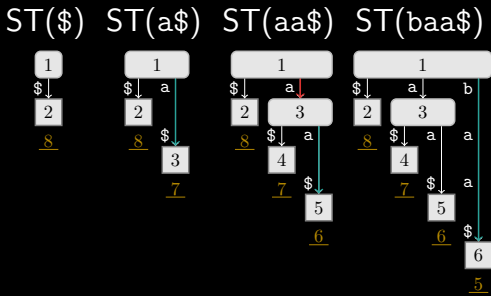
output: suffix tree $ST(T)$ of T

goal: construct $ST(T[j..])$ from $ST(T[j+1..])$ as $T[j]$ arrives, for $j = n, n-1, \dots, 1$

Example

($T = \text{aababaa}\$$)

- ▮ start with $\$$
- ▮ insert $a\$$
- ▮ insert $aa\$$
- ▮ insert $baa\$$

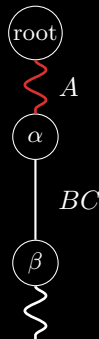


Insertion Point

where do we update ST when inserting $T[j..]$ into $ST(T[j + 1..])$?

Naive solution

1. Search longest path from root in $ST(T[j + 1..])$ matching $T[j..]$

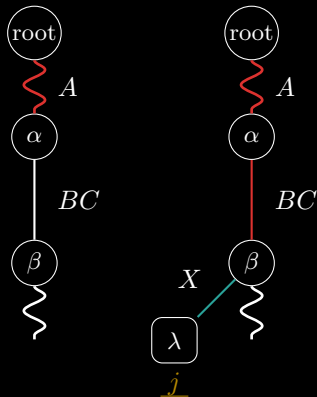


Insertion Point

where do we update ST when inserting $T[j..]$ into $ST(T[j + 1..])$?

Naive solution

1. Search longest path from root in $ST(T[j + 1..])$ matching $T[j..]$
2. If $T[j..] = ABCX$, then branch off at node β



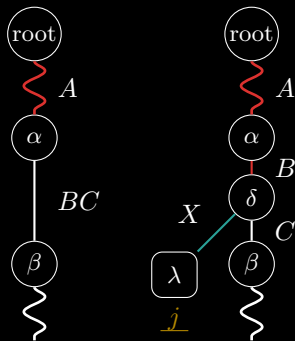
Insertion Point

where do we update ST when inserting $T[j..]$ into $ST(T[j + 1..])$?

Naive solution

1. Search longest path from root in $ST(T[j + 1..])$ matching $T[j..]$
2. If $T[j..] = ABCX$, then branch off at node β
3. If $T[j..] = ABX$, then
 - 3.1 split edge (α, β) to create node δ
 - 3.2 branch off at node δ

In any case, we call the branch-off locus the **insertion point**.



Insertion Point Problem

Key observation

- suffix $T[j..]$ is represented by a **leaf** λ in $ST(T[j..])$
- to update $ST(T[j + 1..])$ to $ST(T[j..])$, we need to insert λ
- this requires finding the **insertion point** for $T[j..]$ in $ST(T[j + 1..])$

Problem (INSERTIONPOINT)

Input: suffix tree $ST(T[j + 1..])$ and letter $T[j]$.

Output: the locus of the longest prefix of $T[j..]$ appearing in $T[j + 1..]$.

Let t_{SU} denote the time complexity to solve INSERTIONPOINT.

Time Complexity t_{SU}

naive approach:

- ▀ traverse $\text{ST}(T[j + 1..])$ from root to match $T[j..]$
- ▀ $O(n)$ time

Can we do better?

t_{SU}	Reference
$O(\lg \sigma)$ amortized	Weiner'73
$O(\lg n)$	Amir, Kopelowitz, +'05
$O(\sigma \log \log n)$	Breslauer, Italiano'13
$O(\log \log n + \log \log \sigma)$ expected	Kopelowitz'12
$O(\log \log n + \frac{\log^2 \log \sigma}{\log \log \log \sigma})$	Fischer, Gawrychowski'15
$O(\log \log n)$ for $\sigma = O(\log^{1/4} n)$	Kuchеров, Nekrich'17

About Insertion Point

What information does INSERTIONPOINT store?

- ▀ the longest common prefix between $T[j..n]$ and the suffix in $ST(T[j + 1..n])$ that is lexicographically closest to $T[j..n]$

Can solve longest repeating prefix problem!

Problem (LONGESTREPEATINGPREFIX)

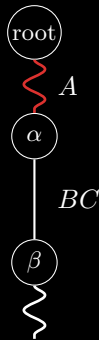
Input: suffix tree $ST(T[j + 1..])$ and letter $T[j]$.

Output: length of the longest repeating prefix of $T[j..]$.

solving LONGESTREPEATINGPREFIX via INSERTIONPOINT

algorithm

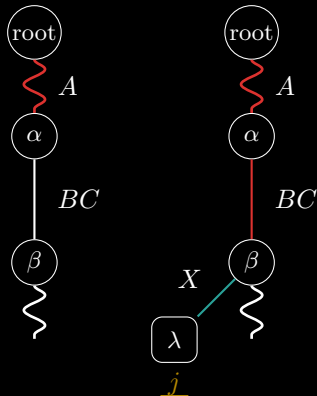
1. Compute INSERTIONPOINT for $T[j..]$ in $ST(T[j + 1..])$



solving LONGESTREPEATINGPREFIX via INSERTIONPOINT

algorithm

1. Compute INSERTIONPOINT for $T[j..]$ in $ST(T[j+1..])$
2. If INSERTIONPOINT is β return $|ABC|$



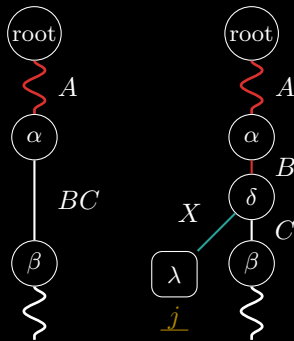
solving LONGESTREPEATINGPREFIX via INSERTIONPOINT

algorithm

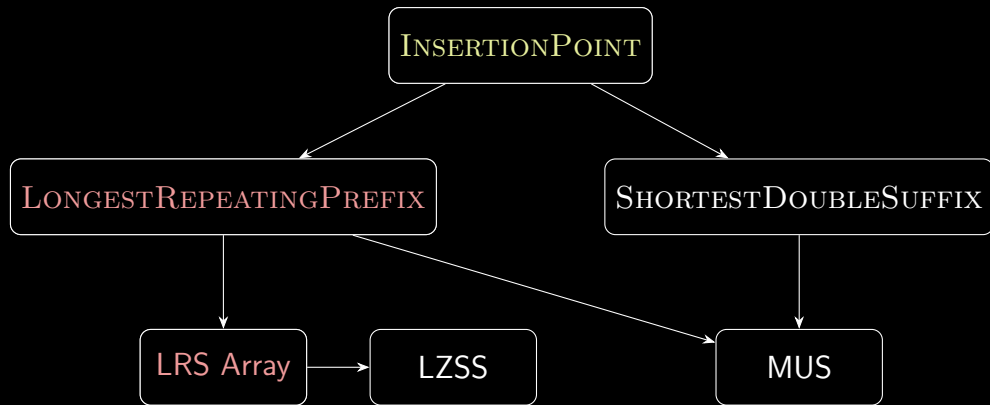
1. Compute INSERTIONPOINT for $T[j..]$ in $ST(T[j + 1..])$
2. If INSERTIONPOINT is β return $|ABC|$
3. If INSERTIONPOINT is δ return $|AB|$

In any case: output string depth of
INSERTIONPOINT

What else can we do with it?



Longest Repeating Suffix Array



Longest Repeating Suffix Array

with LONGESTREPEATINGPREFIX we can compute:

Definition (longest repeating suffix array (LRS))

The longest repeating suffix array (LRS) of a string $T[1..n]$ is an array $LRS[1..n]$ such that for each position $j \in [1..n]$, $LRS[j]$ is the length of the longest suffix of $T[1..j]$ that occurs at least twice in $T[1..j]$.

known solutions to compute LRS online, time is per letter

time	reference
$O(\log^3 n)$	Okanohara, Sadakane'08
$O(\log^2 n)$ amortized	Prezza, Rosone'20
$O(t_{SU})$	this work

Near-real-time LRS computation

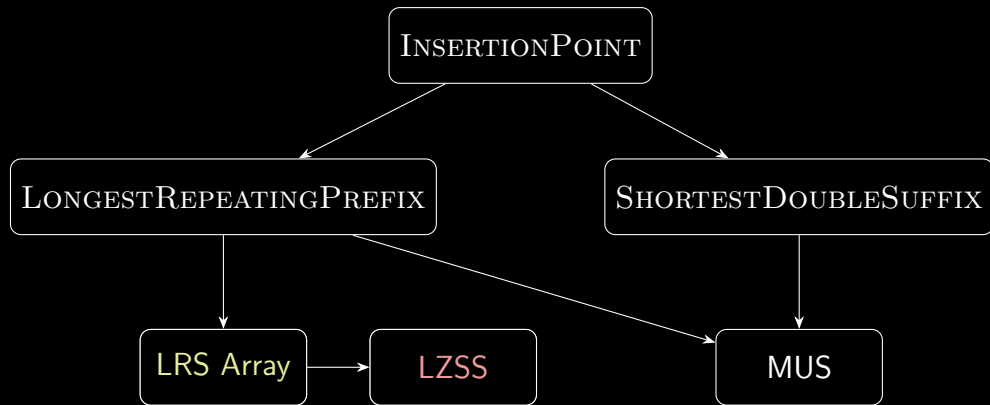
Follow ideas of Amir, Landau, + '02:

- ▮ compute ST on reversed text $\overleftarrow{T} = T[n]T[n-1] \cdots T[1]$ online
- ▮ use LONGESTREPEATINGPREFIX queries to compute LRS

Theorem

can compute LRS online in $O(t_{\text{SU}})$ worst-case time per letter.

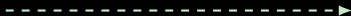
LZSS computation



Lempel–Ziv–Storer–Szymanski (LZSS)

$T = a a b a b a a \$$

1 2 3 4 5 6 7 8



符号化：

- ▮ Replace longest possible read substring with back-reference

Lempel–Ziv–Storer–Szymanski (LZSS)

$T =$ **a** a b a b a a \$

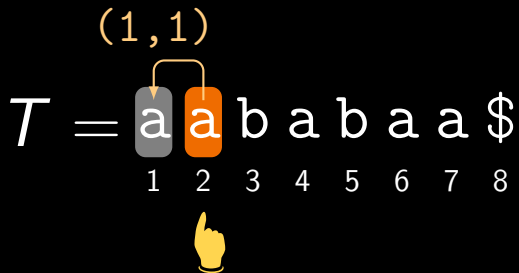
1 2 3 4 5 6 7 8



符号化 : a

- ▮ Replace longest possible read substring with back-reference
- ▮ Start with a single letter (no replacement)

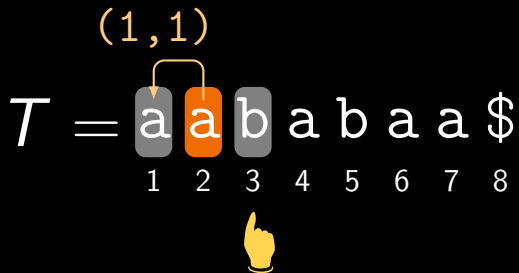
Lempel–Ziv–Storer–Szymanski (LZSS)



符号化 : $a(1, 1)$

- ▮ Replace longest possible read substring with back-reference
- ▮ Start with a single letter (no replacement)
- ▮ Replace with the longest previous occurrence (overlaps are allowed)

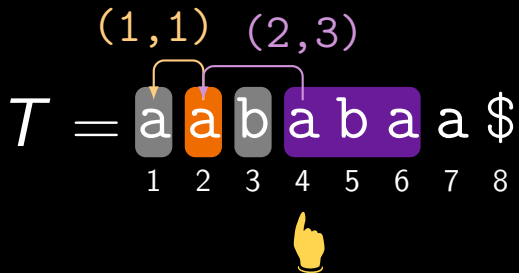
Lempel–Ziv–Storer–Szymanski (LZSS)



符号化 : $a(1, 1)b$

- ▮ Replace longest possible read substring with back-reference
- ▮ Start with a single letter (no replacement)
- ▮ Replace with the longest previous occurrence (overlaps are allowed)

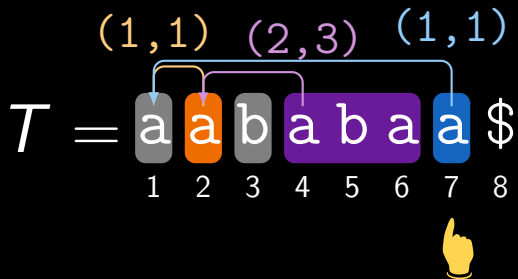
Lempel–Ziv–Storer–Szymanski (LZSS)



符号化 : $a(1, 1)b(2, 3)$

- ▀ Replace longest possible read substring with back-reference
- ▀ Start with a single letter (no replacement)
- ▀ Replace with the longest previous occurrence (overlaps are allowed)

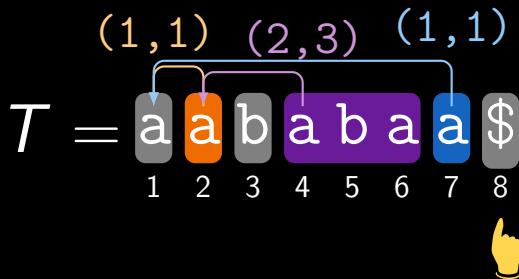
Lempel–Ziv–Storer–Szymanski (LZSS)



符号化 : $\text{a}(1, 1)\text{b}(2, 3)(1, 1)$

- ▀ Replace longest possible read substring with back-reference
- ▀ Start with a single letter (no replacement)
- ▀ Replace with the longest previous occurrence (overlaps are allowed)

Lempel–Ziv–Storer–Szymanski (LZSS)



符号化 : $a(1,1)b(2,3)(1,1)\$$

- ▮ Replace longest possible read substring with back-reference
- ▮ Start with a single letter (no replacement)
- ▮ Replace with the longest previous occurrence (overlaps are allowed)

LZSS computation

known solutions to compute LZSS online, time is per letter:

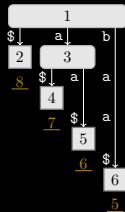
time	reference
$O(\log \sigma)$ amortized	Gusfield'97
$O(t_{\text{SU}})$	this work
$O(\log^3 n)$	Okanohara, Sadakane'08
$O(\log^2 n)$ amortized	Starikovskaya'12
$O(\log n)$ amortized	Yamamoto+'14

note: latter solutions use compact space

Near-real-time LZSS computation

- suppose: $j = 3$, j.e, have processed $T[1..3] = \text{aab}$ of $T = \text{aababaa}$
- have $ST(\overleftarrow{T[1..3]}) = ST(\text{baa})$
- know that next factor F starts at $T[4..] = \text{abaa}$

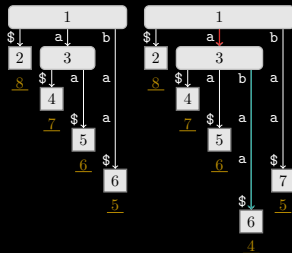
goal: compute F 's length starting at $p = 4$



Near-real-time LZSS computation

goal: compute F 's length starting at
 $p = 4$

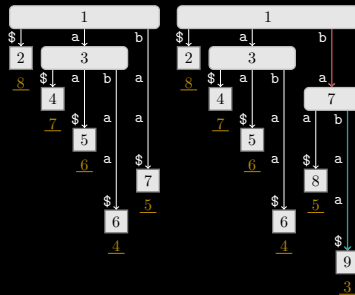
- update $j \leftarrow 4$ to
 $\overleftarrow{\text{ST}(T[1..j])} = \text{ST}(\text{abaa})$
- insertion point has locus $L = a$
- since $|L| \geq p + 1 - j = 1$: **continue**



Near-real-time LZSS computation

goal: compute F 's length starting at
 $p = 4$

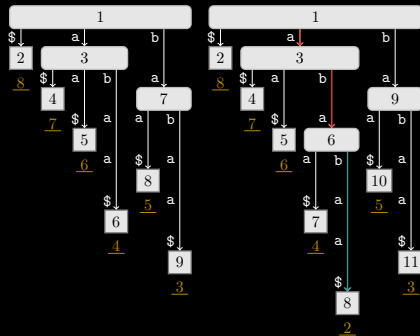
- update $j \leftarrow 5$ to
 $\overleftarrow{\text{ST}(T[1..j])} = \text{ST}(\text{babaa})$
- insertion point has locus $L = \text{ba}$
- since $|L| \geq p + 1 - j = 2$: **continue**



Near-real-time LZSS computation

goal: compute F 's length starting at
 $p = 4$

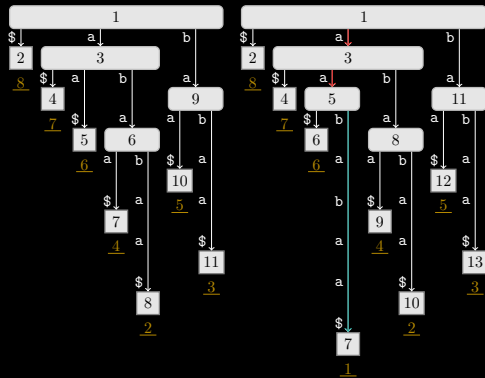
- update $j \leftarrow 6$ to
 $\overleftarrow{\text{ST}(T[1..j])} = \text{ST}(\text{ababaa})$
- insertion point has locus $L = \text{aba}$
- since $|L| \geq p + 1 - j = 3$: **continue**



Near-real-time LZSS computation

goal: compute F 's length starting at
 $p = 4$

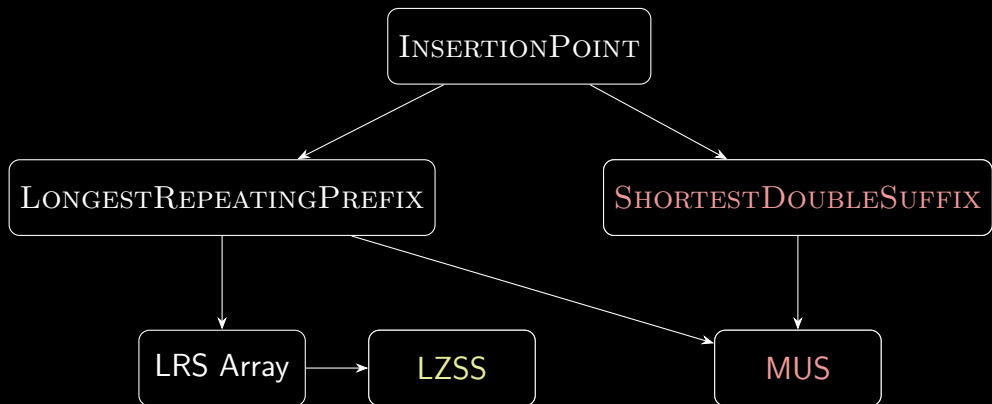
- update $j \leftarrow 7$ to
 $\overleftarrow{\text{ST}(T[1..j])} = \text{ST}(\text{aababaa})$
- insertion point has locus $L = \text{aa}$
- since $|L| < p + 1 - j = 4$: **stop**
- know now: $|F| = p - j = 3$



LZSS computation (cont.)

Theorem

can compute LZSS factorization of T online in $O(t_{\text{SU}})$ worst-case time per letter.



Minimum Unique Substring (MUS)

with LONGESTREPEATINGPREFIX we can compute:

Definition (minimum unique substring (MUS))

of a string $T[1..n]$ is a substring $T[\ell..r]$ such that

- ▮ $T[\ell..r]$ occurs exactly once in T
- ▮ $T[\ell + 1..r]$ and $T[\ell..r - 1]$ occurs at least twice in T (if $\ell < r$)

Let $\text{MUS}(T[1..j])$ denote the set of all MUSs of $T[1..j]$.

Example ($T[1..4] = \text{aaba}$ is a MUS)

- ▮ $T[1..4] = \text{aaba}$ occurs once
- ▮ $T[1..3] = \text{aab}$ occurs twice
- ▮ $T[2..4] = \text{aba}$ occurs twice

$T = \text{a a b a b a a b}$

$\text{MUS}(T) = \{\text{abaa}, \text{bab}, \text{baa}\}$

existing work

- known solutions to compute MUS
online, time is per letter

time	reference
$O(\lg \sigma)$ amortized	Mieno+'20
$O(t_{\text{SU}})$	this work

growable array

is an integer array $A[1..n]$ of n elements
supporting

- $A.\text{grow}()$: increment size of A
- $A[j]$: random read/write access to
entry $A[j]$

growable arrays:

time	reference
$O(1)$ amortized	array doubling
$O(1)$	Brodnik+'99

approach of Mieno+'20

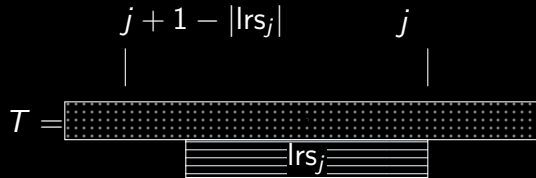
showed that $\text{MUS}(T[1..j])$ can be computed from $\text{MUS}(T[1..j-1])$, lrs_j and sds_j

Definition (shortest double suffix sds_j)

is the shortest suffix of $T[1..j]$ with exactly two occurrences in $T[1..j]$, which may not exist.

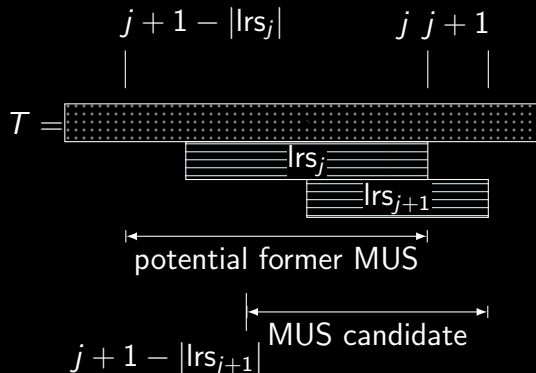
online MUS algorithm

1. Read new letter $T[j+1]$



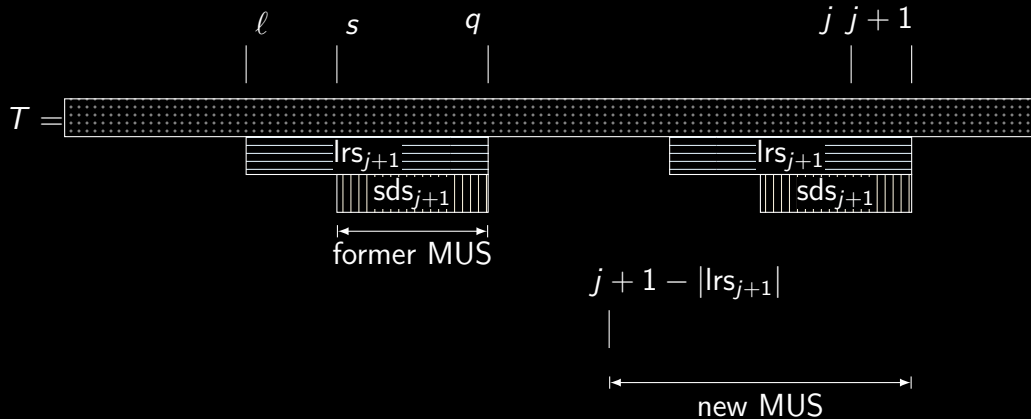
online MUS algorithm

1. Read new letter $T[j+1]$
 2. Add new MUS
 $[j+1 - |lrs_{j+1}|..j+1]$ if
 $|lrs_{j+1}| \leq |lrs_j|$
- ▀ If sds_{j+1} does not exist, we are done
 - ▀ From now on: assume that sds_{j+1} exists



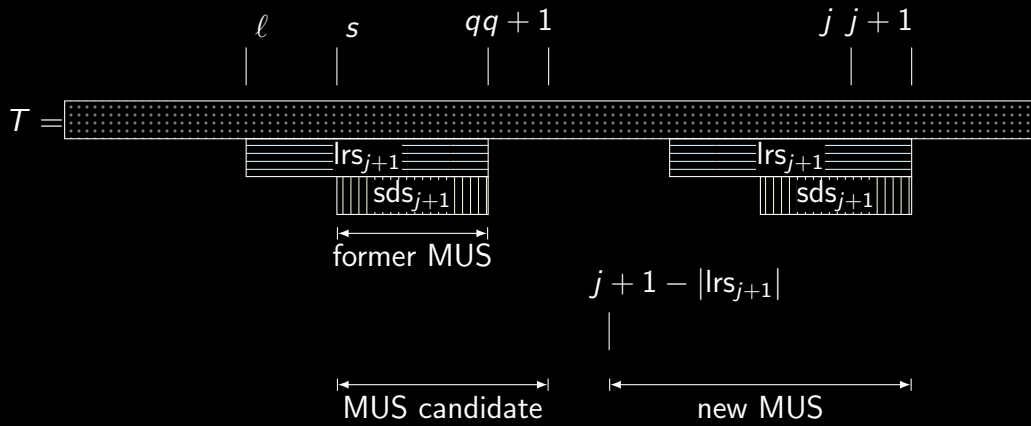
Case: sds_{j+1} exists

- ▀ find sds_{j+1} 's non-suffix occurrence $[s..q]$:
 - s : the label of the leaf of the locus of sds_{j+1} that already existed in $\text{ST}(\overleftarrow{T[1..j]})$
 - $q = s + |\text{sds}_{j+1}| - 1$



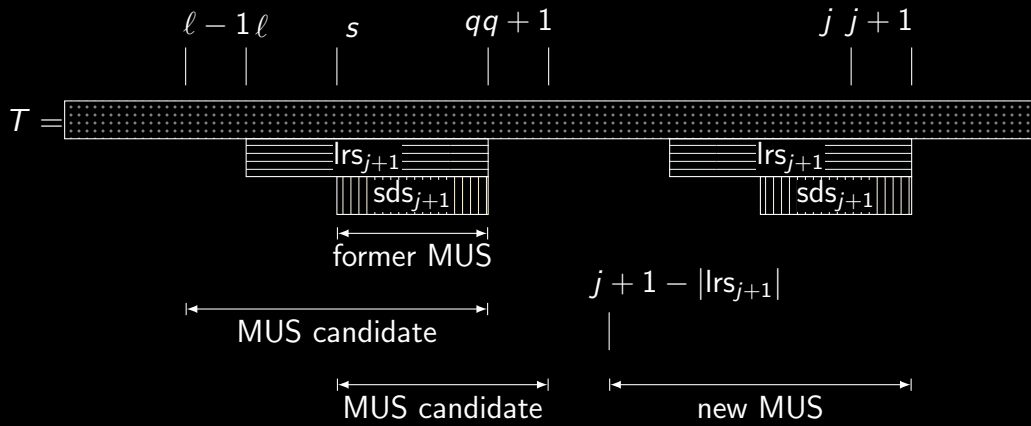
Case: sds_{j+1} exists

- delete former MUS $[s..q]$
- if there is no MUS ending at position $q + 1$: add MUS $[s..q + 1]$



Case: sds_{j+1} exists

- delete former MUS $[s..q]$
- if there is no MUS ending at position $q + 1$: add MUS $[s..q + 1]$
- if $q - |lrs_{j+1}| \geq 1$ and $\nexists$ MUS starting at position $q - |lrs_{j+1}|$: add MUS $[q - |lrs_{j+1}|..q]$



Reduction from INSERTIONPOINT

Problem (SHORTESTDOUBLE_SUFFIX sds_j)

Input: suffix tree $ST(\overleftarrow{T[1..j]})$ and letter $T[j+1]$.

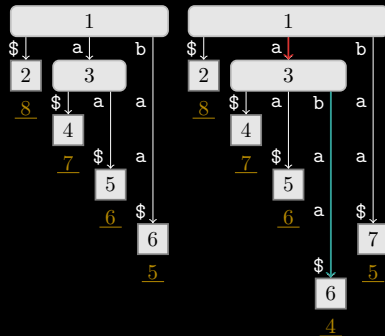
Output: the shortest prefix of $\overleftarrow{T[1..j+1]}$ that has exactly one occurrence in $\overleftarrow{T[1..j]}$, if it exists.

can solve SHORTESTDOUBLE_SUFFIX via INSERTIONPOINT!

Reduction from INSERTIONPOINT

algorithm

if insertion point is an already existing node in $ST(\mathcal{T}[j+1..])$:
LONGESTREPEATINGPREFIX occurs at least three times in $\mathcal{T}[j..]$ and sds_j cannot exist



▮ $T = aababaa$

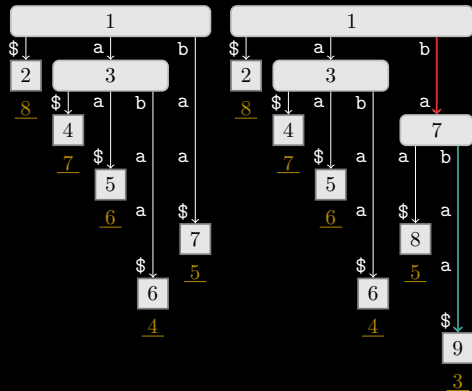
▮ Update from $ST(baa)$ to $ST(abaa)$

Reduction from INSERTIONPOINT

algorithm

else:

- LONGESTREPEATINGPREFIX becomes a node in $ST(T[j..])$ with exactly two leaves
- sds_j exists and its locus is located on the parent edge to the locus of LONGESTREPEATINGPREFIX



- $T = aababaa$
- Update from $ST(abaa)$ to $ST(babaa)$

conclusion

Theorem

can compute $\text{MUS}(T[1..j])$ online in $O(t_{\text{SU}})$ worst-case time per letter.

summary

- ▮ real-time ST construction remains open, but possible in near-real-time
- ▮ many applications (LRS, LZSS, MUS, ...) solvable in near-real-time via `INSERTIONPOINT`
- ▮ not shown: near-real-time computation of reversed LZ factorization / suffixient sets

Any questions are welcome!